

SwiftLoad: Multithreaded **File Downloader Project** **Report**

Course Name: PCCCS495 - Term II Project

Student Name: Prithvi Raj Banik

Enrollment : 12024052019013

Roll Number: 14

Semester: Spring 2026

Problem Statement

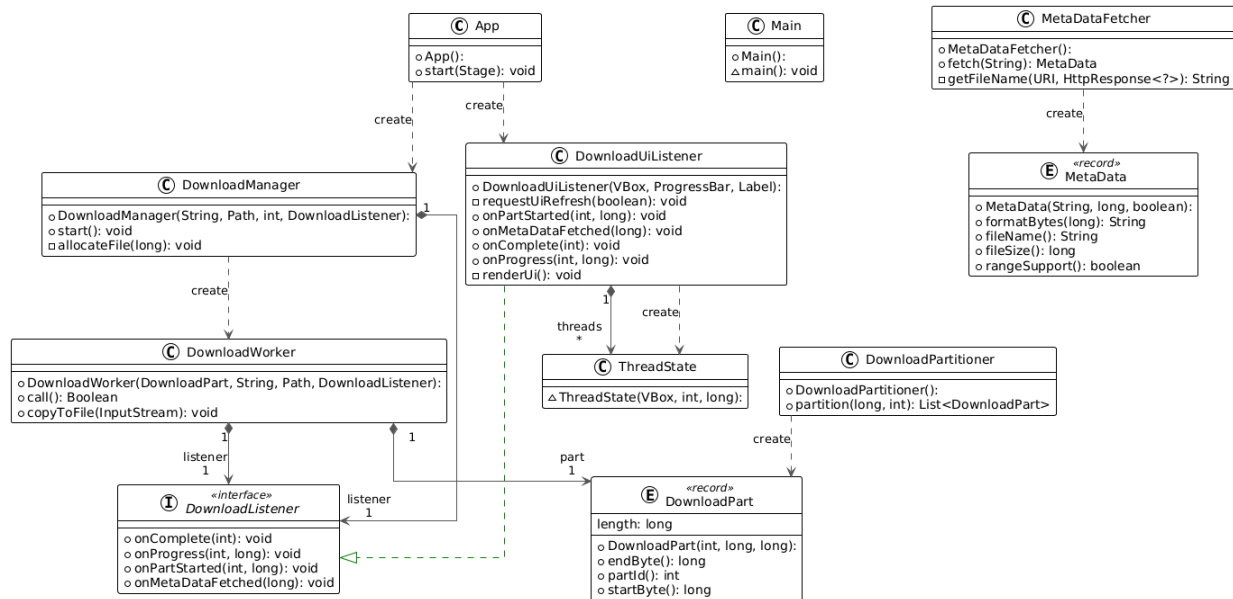
In the modern digital age, the ability to efficiently retrieve data may not be restricted by the actual limitations of the network, but by the architectural limitations of the client-side software itself. Many conventional download managers employ a single-threaded technique in retrieving files via the HTTP or HTTPS protocol.

There are a number of major disadvantages to the single-threaded technique:

1. **Underutilization of Bandwidth:** Modern server software allows for a number of simultaneous connections, but single threaded clients only employ a single connection at a time.
2. **UI Blocking and Responsiveness:** In many conventional implementations, network I/O and file I/O happen on the same thread as the user interface, causing the interface to appear frozen or unresponsive.
3. **Lack of Granular Control:** There is no information provided to the end user on the manner in which different parts of a file are being downloaded or where the actual problems may be occurring..

System Design

UML Class Diagram:



Description of Major Classes:

- **gui/App**: This class acts as the entry point for the JavaFX program. It controls the primary stage, validates user input for URLs and thread counts, and initializes the high-level orchestration of the download.
- **core/DownloadManager**: This class acts as the brain of the program, controlling the lifecycle of the download, including retrieving the metadata, pre-allocating the file on disk, and managing the thread pool.
- **core/DownloadWorker**: This class implements the `Callable` interface. It makes the actual HTTP Range request and writes the byte stream to a precise offset in the destination file.

- **util/MetaDataFetcher:** This class interacts with the remote server to retrieve information about the file, including size, name, and most importantly, whether the server allows Range requests.nt (Range requests).
- **gui/DownloadUiListener:** This class implements the observer pattern and acts as a mediator between the worker threads and the JavaFX UI elements, safely updating the UI.

Package Structure:

The project follows a clean, package-based organization:

- **com.xolo.swiftload.core:** Execution logic.
- **com.xolo.swiftload.gui:** Controllers, views, and UI specific state management.
- **com.xolo.swiftload.models:** Records and interfaces that define the system's data contracts.
- **com.xolo.swiftload.util:** Helper classes for networking and file partitioning.

Interaction Flow

1. **Input:** User enters a URL and thread count.
2. **Metadata Fetch:** The server headers are queried using the MetaDataFetcher by the DownloadManager..
3. **Partitioning:** DownloadPartitioner calculates start and end byte ranges for each thread.
4. **Allocation:** A RandomAccessFile is created and expanded to the full file size immediately to reserve space.
5. **Concurrent Download:** Multiple DownloadWorker objects are

created and submitted to the ExecutorService..

6. **Progress Reporting:** The workers notify the DownloadUiListener, and the speed is calculated and displayed in the progress bars.

OOP Concepts Demonstrated

Inheritance (Implementation):

Where : In the relationship of gui/DownloadUiListener and models/DownloadListener interface.

Reason: Inheritance, in this case, is used to enforce a strict "is a" relationship. By having the UI listener implement this interface, it is forced to inherit a requirement to implement concrete logic for the progress methods that are defined by the core engine.

```
public interface DownloadListener { 13 usages 2 implementations Xolo978
    default void onPartStarted(int partId, long partSize) {} 1 usage 1 override Xolo978
    void onProgress(int partId, long bytesDownloaded); 1 usage 2 implementations Xolo978
    void onMetaDataFetched(long fileSize); 1 usage 2 implementations Xolo978
    void onComplete(int partId); 1 usage 2 implementations Xolo978
}
```

Polymorphism

Where: In the implementation of DownloadUiListener.

Reason: It allows the application to handle different types of listeners through a common interface. The DownloadManager treats all listeners as DownloadListener objects, regardless of their concrete class.

```

@Override 1 usage  Xolo978
public void onProgress(int partId, long bytesDownloaded) {
    totalDownloaded.addAndGet(bytesDownloaded);
    ThreadState state = threads.get(partId);
    if (state != null) {
        state.downloaded.addAndGet(bytesDownloaded);
    }
    requestUiRefresh(force: false);
}

```

Encapsulation

Where: In the ThreadState class.

Reason: To group all relevant information pertaining to the state of a particular thread's UI, i.e., the progress bar, speed labels, and byte counts, into a manageable unit.

```

public class ThreadState { 5 usages  Xolo978
    final ProgressBar progressBar = new ProgressBar(0); 4 usages
    final Label speedLabel; 5 usages
    final AtomicLong downloaded = new AtomicLong(initialValue: 0); 2 usages
    long downloadedAtLastSample = 0; 2 usages
    final long totalBytes; 5 usages

    ThreadState(VBox! container, int partId, long totalBytes) { 1 usage  Xolo978
        this.totalBytes = totalBytes;
        progressBar.setPrefWidth(160);
        speedLabel = new Label(text: "0.00 MB/s");
        Label nameLabel = new Label(text: "Thread " + partId);
        container.getChildren().add(new HBox(spacing: 10, nameLabel, progressBar, speedLabel));
    }
}

```

Implementation Highlights

Safe Parallel File Writes:

One of the most complex parts of the project was ensuring multiple threads could write to the same file simultaneously without data corruption.

```
public void copyToFile(InputStream input) throws IOException { 1 usage  Xolo978 *
    try(RandomAccessFile file = new RandomAccessFile(savePath.toFile(), mode: "rw")){
        file.seek(part.startByte());
        byte[] buffer = new byte[8192];
        int i;
        while((i =input.read(buffer))!=-1){
            file.write(buffer, off: 0,i);
            if(listener!=null){
                listener.onProgress(part.partId(),i);
            }
        }
        if(listener!=null){
            listener.onComplete(part.partId());
        }
    }
}
```

Justification: We use the RandomAccessFile's seek() method to position the file pointer at the specific startByte of each partition. This ensures that even though threads are writing to the same file, they are operating in entirely unique disk segments.

Adaptive Multithreading Fallback:

Not all servers support range requests. The system must detect this and degrade gracefully.

```
URI uri = URI.create(url);
HttpRequest req;
req = HttpRequest.newBuilder()
    .uri(uri)
    .HEAD()
    .header(name: "User-Agent", value: "Mozilla/5.0")
    .build();
HttpResponse<Void> res;
try{
    res = client.send(req, HttpResponse.BodyHandlers.discarding());
}catch(IOException e){
    req = HttpRequest.newBuilder()
        .uri(uri)
        .header(name: "User-Agent", value: "Mozilla/5.0")
        .header(name: "Range", value: "bytes=0-1")
        .GET()
        .build();
    res = client.send(req, HttpResponse.BodyHandlers.discarding());
}

long size = res.headers() HttpHeaders
    .firstValueAsLong(name: "Content-Length") OptionalLong
    .orElse(other: -1L);
//Content range is returned when using get request
```

```
long size = res.headers() HttpHeaders
    .firstValueAsLong(name: "Content-Length") OptionalLong
    .orElse(other: -1L);
//Content range is returned when using get request
if(size<=2){
    String range = res.headers() HttpHeaders
        .firstValue(name: "Content-Range") Optional<String>
        .orElse(other: "");
    if (range.contains("/")) {
        size = Long.parseLong(range.split(regex: "/")[1]);
    }
}
String acceptRange = res.headers() HttpHeaders
    .firstValue(name: "Accept-Ranges") Optional<String>
    .orElse(other: "") String
    .toLowerCase();
```


Testing & Error Handling

Test Cases Considered:

- **Variable Thread Counts:** Tested that the partitioning logic is correct for prime numbers of threads and large thread counts (e.g 32 threads).
- **Small vs Large Files:** Test cases include small text files (1 KB) and large binary files (100 MB).
- **Redirect Handling:** Verifies that the MetadataFetcher follows redirects to determine the location of the final resource.

Edge Cases Handled:

- **Missing Content-Length:** If not specified in the server, the default implementation will be single-threaded streaming..
- **Invalid File Names:** Sanitizing URLs to extract valid filenames (e.g., stripping query parameters like ?v=123).

Failure Scenarios:

- **Connection Timeout:** If the worker loses the connection, it goes into a failed state, which alerts the user..
- **Partial Success:** If some threads are completed but others are not, the system will indicate to the user which parts are missing

Git Workflow

Commit History:

```
* f9e21e5 (HEAD -> main, origin/main, origin/HEAD) feat: Add download folder picker
* e2e2dd2 Dynamic thread slider based on available system resources
* e5b223b Refactored DownloadListener to use less space
* 9e0e852 feat: Add per thread based download details
* 76bf1ed feat: Add interactive gui for downloading file
* 8969fb8 Added basic JAVAFX ui and configuration
* 4ec03f1 Implemented the observer in DownloadManager
* 0b1d609 feat: Add fallback GET method for fetching meta data
* b474adc feat: Added DownloadManager for completely downloading a file
* 66f1496 Add range support checker
* e22074e feat: Implemented DownloadWorker implementing Callable
* 14af715 Data models relocated and Partitioner was added
* 68afe2e Added missing tests for DataFetcher and HttpClient timeout
* a189315 feat: Added metadata fetching from url and tests
* 0b0edc2 feat: Added file metadata retrieval
* c167e62 Added gradle build script and basic setup
* 27d78b2 add deadline
* 59939a0 Initial commit
```

Development Progression:

1. **Initial Setup:** Establishing the Gradle project and JavaFX boilerplate.
2. **Core Utilities:** Writing the MetaDataFetcher and Partitioner.
3. **Concurrency Engine:** Developing the DownloadManager and worker thread logic.
4. **UI Integration:** Connecting the core to the JavaFX stage via listeners.
5. **Refinement:** Implementing speed calculations, folder selection, and error handling.

Conclusion & Future Scope

SwiftLoad demonstrates the effectiveness of using Java's high-level concurrency APIs to solve real-world network performance problems. By partitioning downloads and isolating I/O from the UI, the application provides a high-performance experience that rivals professional download managers.

Future Scope

- **Pause and Resume:** Saving the current byte ranges to a .json metadata file to allow resuming after a program restart.
- **Dynamic Threading:** Automatically adjusting the number of threads based on live network latency and server response times.
- **Browser Integration:** Developing an API to allow browser extensions to send download links directly to SwiftLoad.